非线性规划课后作业

2352018 刘彦

1.

3.1 某工厂向用户提供发动机，按合同规定，其交货数量和日期是：第一季度末交 40 台，第二季末交 60 台，第三季末交 80 台。工厂的最大生产能力为每季 100 台，每季的生产费用是 $f(x)=50x+0.2x^2$（元），此处 x 为该季生产发动机的台数。若工厂生产得多，多余的发动机可移到下季向用户交货，这样，工厂就需支付存储费，每台发动机每季的存储费为 4 元。问该厂每季应生产多少台发动机，才能既满足交货合同，又使工厂所花费的费用最少（假定第一季度开始时发动机无存货）？

⑴变量定义

delivery：交货需求向量，表示三个季度的需求分别为 40、60 和 80 件。

production_capacity：每季度生产能力的上限，设为 100 件。

storage_cost_per_unit：每件产品每季度的存储成本，设为 4 元。

production_cost_coefficients：生产成本函数的系数，形式为$c(x)=50x+0.2x^2$，其中 50 是线性项系数，0.2 是二次项系数。

此外，隐含的决策变量是$x=[x_1,x_2,x_3]$，表示三个季度的生产量。

⑵目标函数

目标是最小化总成本，包括生产成本和存储成本。

完整目标函数表达式：

$$\min z=\frac{1}{2}x^THx+f^Tx$$

系数计算说明

- H是二次项系数矩阵，来源于生产成本的二次项
  - $H=diag(2\times0.2,2\times0.2,2\times0.2)$
  - 因子2是因为二次规划标准形式中二次项前有$\frac{1}{2}$
- 是线性项系数向量，包含生产成本的线性项和存储成本的影响。
  - 生产成本线性项：$50x_i$
  - 存储成本：与库存量相关，库存为生产量减去交货需求的累积差。
  - $f=[50+4\times3,50+4\times2,50+4\times1]$，这里[3,2,1]表示各季度库存对后续季度的累计影响。

⑶约束条件

- 交货需求
  - 第 1 季度的要求：$x_1\geq40$
  - 第 2 季度的要求：$x_1+x_2\geq100$

- 第 3 季度的要求：$x_1 + x_2 + x_3 \geq 180$

- 产能约束
  - 第 1 季度的产能：$0 \leq x_1 \leq 100$
  - 第 2 季度的产能：$0 \leq x_2 \leq 100$
  - 第 3 季度的产能：$0 \leq x_3 \leq 100$

⑷求解与输出

调用 quadprog 函数求解模型，输出最优解（各变量值）和目标函数值（总费用）。

⑸代码实现和输出

```
clc; clear;
% 交货需求
delivery = [40; 60; 80]; % 每季度的交货量
production_capacity = 100; % 每季度的最大生产能力
storage_cost_per_unit = 4; % 每台发动机每季度的存储费用
production_cost_coefficients = [50, 0.2]; % 生产费用系数 [线性项, 二次项]
% **计算存储成本的固定部分**
% 假设每季度生产量分别为 x1, x2, x3:
% 存储量计算公式: 库存 = 生产 - 交货（累积计算）
storage_cost_fixed = sum(storage_cost_per_unit * cumsum(delivery));
% **二次规划参数**
H = diag(2 * production_cost_coefficients(2) * ones(3, 1)); % 二次项系数矩阵
f = production_cost_coefficients(1) + storage_cost_per_unit * [3; 2; 1]; %
线性项系数
% **约束条件**
A = [-1  0  0;           % x1 ≥ 40
     -1 -1  0;           % x1 + x2 ≥ 100
     -1 -1 -1;           % x1 + x2 + x3 ≥ 180
      1  0  0;           % x1 ≤ 100
      0  1  0;           % x2 ≤ 100
      0  0  1];          % x3 ≤ 100
b = [-40; -100; -180; 100; 100; 100];
% 变量上下界
lb = zeros(3, 1); % 生产量下限
ub = production_capacity * ones(3, 1); % 生产量上限
% **求解二次规划问题**
[x_opt, fval] = quadprog(H, f, A, b, [], [], lb, ub);
% **计算总费用**
total_cost = fval - storage_cost_fixed;
% **显示结果**
disp('各季度最优生产量: ');
disp(['x1 = ', num2str(x_opt(1)), ' 件']);
```

```
disp(['x2 = ', num2str(x_opt(2)), ' 件']);
disp(['x3 = ', num2str(x_opt(3)), ' 件']);
disp(['计算得出存储费用固定部分: ', num2str(storage_cost_fixed), ' 元']);
disp(['总费用: ', num2str(total_cost), ' 元']);
```

```
各季度最优生产量:
x1 = 50 件
x2 = 60 件
x3 = 70 件
计算得出存储费用固定部分: 1280 元
总费用: 11280 元
```

2.

3.2 用 Matlab 的非线性规划命令 fmincon 求解飞行管理问题的模型二。

例 3.10(本题选自 1995 年全国大学生数学建模竞赛 A 题) 在约 10000m 高空的某边长 160km 的正方形区域内,经常有若干架飞机水平飞行。区域内每架飞机的位置和速度向量均由计算机记录其数据,以便进行飞行管理。当一架欲进入该区域的飞机到达区域边缘时,记录其数据后,要立即计算并判断是否会与区域内的飞机发生碰撞。如果会碰撞,则应计算如何调整各架(包括新进入的)飞机飞行的方向角,以避免碰撞。现假定条件如下:

(1) 不碰撞的标准为任意两架飞机的距离大于 8km;

(2) 飞机飞行方向角调整的幅度不应超过 30°;

(3) 所有飞机飞行速度均为 800km/h;

(4) 进入该区域的飞机在到达区域边缘时,与区域内飞机的距离应在 60km 以上;

(5) 最多需考虑 6 架飞机;

(6) 不必考虑飞机离开此区域后的状况。

请你对这个避免碰撞的飞行管理问题建立数学模型,列出计算步骤,对以下数据进行计算(方向角误差不超过 0.01°),要求飞机飞行方向角调整的幅度尽量小。

设该区域 4 个顶点的坐标为(0,0),(160,0),(160,160),(0,160)。飞行记录数据见表 3.7。

表 3.7 飞行记录数据

飞机编号	横坐标 x	纵坐标 y	方向角/(°)
1	150	140	243
2	85	85	236
3	150	155	220.5
4	145	50	159
5	130	150	230
新进入	0	0	52

注 3.3 方向角指飞行方向与 x 轴正向的夹角。

1. 符号说明

a 为飞机飞行速度,$a=800$km/h;

(x_i^0, y_i^0) 为第 i 架飞机的初始位置($i=1,2,\cdots,6$),$i=6$ 对应新进入的飞机;

$(x_i(t), y_i(t))$ 为第 i 架飞机在 t 时刻的位置;

θ_i^0 为第 i 架飞机的原飞行方向角,即飞行方向与 x 轴夹角,$0\leqslant\theta_i^0<2\pi$;

$\Delta\theta_i$ 为第 i 架飞机的方向角调整量,$-\frac{\pi}{6}\leqslant\Delta\theta_i\leqslant\frac{\pi}{6}$;

$\theta_i=\theta_i^0+\Delta\theta_i$ 为第 i 架飞机调整后的飞行方向角。

2）模型二

两架飞机 i,j 不发生碰撞的条件为

$$[x_i(t)-x_j(t)]^2+[y_i(t)-y_j(t)]^2\geqslant 64,$$
$$1\leqslant i\leqslant 5,\quad i+1\leqslant j\leqslant 6,\quad 0\leqslant t\leqslant \min\{T_i,T_j\},$$

式中：T_i,T_j 分别表示第 i,j 架飞机飞出正方形区域边界的时刻。这里

$$x_i(t)=x_i^0+at\cos\theta_i,\quad y_i(t)=y_i^0+at\sin\theta_i,\quad i=1,2,\cdots,n,$$
$$\theta_i=\theta_i^0+\Delta\theta_i,\quad |\Delta\theta_i|\leqslant\frac{\pi}{6},\quad i=1,2,\cdots,n.$$

下面我们把约束条件式(3.23)加强为对所有的时间 t 都成立，记

$$l_{ij}=[x_i(t)-x_j(t)]^2+[y_i(t)-y_j(t)]^2-64=\tilde{a}_{ij}t^2+\tilde{b}_{ij}t+\tilde{c}_{ij},$$

式中

$$\tilde{a}_{ij}=4a^2\sin^2\frac{\theta_i-\theta_j}{2},$$
$$\tilde{b}_{ij}=2a\{[x_i(0)-x_j(0)](\cos\theta_i-\cos\theta_j)+[y_i(0)-y_j(0)](\sin\theta_i-\sin\theta_j)\},$$
$$\tilde{c}_{ij}=[x_i(0)-x_j(0)]^2+[y_i(0)-y_j(0)]^2-64.$$

则两架 i,j 飞机不碰撞的条件是

$$\Delta_{ij}=\tilde{b}_{ij}^2-4\tilde{a}_{ij}\tilde{c}_{ij}\leqslant 0.$$

建立如下非线性规划模型：

$$\min\sum_{i=1}^{6}(\Delta\theta_i)^2,$$
$$\text{s.t.}\begin{cases}\Delta_{ij}\leqslant 0, & 1\leqslant i\leqslant 5, i+1\leqslant j\leqslant 6,\\ |\Delta\theta_i|\leqslant\dfrac{\pi}{6}, & i=1,2,\cdots,6.\end{cases}$$

⑴参数定义

x0：初始解，一个 6×1 的随机向量，表示 6 个变量的初始调整量，通过 rand(6,1) 生成[0,1]区间内的随机值。

th0：初始角度向量（单位：度），6×1 向量，值为 [243, 236, 220.5, 159, 230, 52]'，表示 6 个对象的初始方向。

th：调整后的角度，计算为$th=th0+x$，其中x是决策变量（调整量，$x=[x_1,x_2,x_3,x_4,x_5,x_6]$）。

xPos：x 坐标向量，6×1 向量，值为[150, 85, 150, 145, 130, 0]'，表示 6 个对象在空间中的 x 位置。

yPos：y 坐标向量，6×1 向量，值为[140, 85, 155, 50, 150, 0]'，表示 6 个对象在空间中的 y 位置。

min_distance：最小不碰撞距离，值为 8。

⑵目标函数

在满足约束的情况下尽量保持初始角度 th0 不变。

$$\min f(x)=\sum_{i=1}^{6}x_i^2$$

⑶约束条件

①边界限制：飞机飞行方向角调整的幅度不应超过 30°

$$-30 \le x_i \le 30 \cdots (i = 1,2,3,4,5,6)$$

②非线性不等式约束

共有 15 个不等式约束，对应 6 个对象两两组合（$C(6,2) = 15$）

假设每个飞机沿θ_i方向有一个“运动轨迹”，我们可以用参数t表示沿方向的位移。例如，对象i的位置可以参数化为：

$$\left(x_i(t), y_i(t)\right) = (x_i + t\cos\theta_i\,, y_i + t\sin\theta_i)$$

两点之间的距离平方函数为：

$$d^2(t) = \left(x_i + t\cos\theta_i - \left(x_j + t\cos\theta_j\right)\right)^2 + \left(y_i + t\sin\theta_i - \left(y_j + t\sin\theta_j\right)\right)^2$$

将距离平方函数写成二次形式：

$$d^2(t) = t^2\left(\cos\theta_i - \cos\theta_j\right)^2 + t^2\left(\sin\theta_i - \sin\theta_j\right)^2 + 2t\left[\left(x_i - x_j\right)\left(\cos\theta_i - \cos\theta_j\right) + \left(y_i - y_j\right)\left(\sin\theta_i - \sin\theta_j\right)\right] + \left(x_i - x_j\right)^2 + \left(y_i - y_j\right)^2$$

使用三角恒等式：

$$\left(\cos\theta_i - \cos\theta_j\right)^2 + \left(\sin\theta_j - \sin\theta_j\right)^2 = 2 - 2\cos\left(\theta_i - \theta_j\right) = 4\sin^2\left(\frac{\theta_i - \theta_j}{2}\right)$$

于是：

$$d^2(t) = 4\sin^2\left(\frac{\theta_i - \theta_j}{2}\right)t^2 + 2t\left[\left(x_i - x_j\right)\left(\cos\theta_i - \cos\theta_j\right) + \left(y_i - y_j\right)\left(\sin\theta_i - \sin\theta_j\right)\right] + \left(x_i - x_j\right)^2 + \left(y_i - y_j\right)^2$$

设$d^2(t) = at^2 + bt + c_0$，$d^2(t) \ge \text{min_distance}^2 = 64$

即$D(t) = at^2 + bt + c_0 - 64 \ge 0$，记$c = c_0 - 64$

这个二次函数的最小值出现在$t = -\frac{b}{2a}(a > 0)$，$\min D = c - \frac{b^2}{4a} \ge 0$

即

$$b^2 - 4ac \le 0$$

其中，$a = 4\sin^2\left(\frac{\theta_i - \theta_j}{2}\right)$，$b = 2\left[\left(x_i - x_j\right)\left(\cos\theta_i - \cos\theta_j\right) + \left(y_i - y_j\right)\left(\sin\theta_i - \sin\theta_j\right)\right]$，$c = \left(x_i - x_j\right)^2 + \left(y_i - y_j\right)^2 - 64$

⑷求解与输出

利用 fmincon 函数求解，输出最优解（6×1 向量）和目标函数的最优值（$f(xOpt) = \sum(xOpt_i)^2$）。

由于x_0是随机的，每次运行结果可能不同。

⑸代码实现和输出

```
clc; clear; close all;
% 定义优化目标函数
objFun = @(x) sum(x.^2);
% 设定初始值和约束
x0 = rand(6,1);
lb = -30 * ones(6,1);
ub = 30 * ones(6,1);
% 调用 fmincon 进行优化
[xOpt, objVal] = fmincon(objFun, x0, [], [], [], [], lb, ub, @conFun);
% 输出结果
disp('Optimal solution:');
disp(xOpt);
disp('Objective function value:');
disp(objVal);
% ---- 局部函数 ----
function [ineq, eq] = conFun(x)
    % 定义非线性不等式约束的函数
    eq = []; % 不存在非线性等式约束
    th0 = [243 236 220.5 159 230 52]'; % 初始角度
    th = th0 + x;                      % 调整后角度
    xPos = [150 85 150 145 130 0]';   % x 坐标
    yPos = [140 85 155 50 150 0]';    % y 坐标
    min_distance = 8;                 % 最小不碰撞距离
    % 计算不等式约束
    ineq = zeros(15,1); % 预分配约束数组
    k = 1;              % 约束索引
    for i = 1:5
        for j = i + 1:6
            a = 4 * (sind((th(i) - th(j)) / 2))^2;
            b = 2 * ((xPos(i) - xPos(j)) * (cosd(th(i)) - cosd(th(j))) + ...
                    (yPos(i) - yPos(j)) * (sind(th(i)) - sind(th(j))));
            c = (xPos(i) - xPos(j))^2 + (yPos(i) - yPos(j))^2 -
min_distance^2;
            ineq(k) = b^2 - 4 * a * c;
            k = k + 1;
        end
    end
end
```

由于x_0是随机的，每次运行结果可能不同，这里举出一个例子：

```
Optimal solution:
   -6.6617
    0.3380
    2.0589
   -0.4990
    6.3380
    1.5705

Objective function value:
   91.6177
```

3.

3.3 用罚函数法求解飞行管理问题的模型二。

⑴参数定义

t0：初始解，一个 6×1 的随机向量，表示 6 个变量的初始调整量，通过 rand(6,1) 生成[0,1]区间内的随机值。

th0：初始角度向量（单位：度），6×1 向量，值为 $[243, 236, 220.5, 159, 230, 52]'$，表示 6 个对象的初始方向。

th：调整后的角度，计算为$th = th0 + t$，其中 t是决策变量（调整量，$t = [t_1, t_2, t_3, t_4, t_5, t_6]$）。

xPos：x 坐标向量，6×1 向量，值为$[150, 85, 150, 145, 130, 0]'$，表示 6 个对象在空间中的 x 位置。

yPos：y 坐标向量，6×1 向量，值为$[140, 85, 155, 50, 150, 0]'$，表示 6 个对象在空间中的 y 位置。

min_distance：最小不碰撞距离，值为 8。

M：初始罚因子，设为 100000，用于惩罚约束违反。

tol：收敛容差，设为 10^{-6}，用于判断约束违反是否足够小。

⑵目标函数

目标函数是最小化调整量的平方和，使用罚函数法将其与约束结合。

目标函数：

$$\min f(t) = \sum_{i=1}^{6} t_i^2$$

带罚项的目标函数：

$$\min penalizedObj(t) = \sum_{i=1}^{6} t_i^2 + M \sum_{k=1}^{15} \max\left(0, g_k(t)\right)^2$$

其中，$g_k(t)$是约束函数返回的第 k 个（共有 15 个，$C(6,2)=15$）不等式约束值，$\max\left(0,g_k(t)\right)^2$表示当约束$g_k(t)\le 0$被违反时，添加一个平方惩罚项，M 是是罚因子，初始值为 100000，逐渐增大以强制约束满足。

⑶约束条件

①边界限制：飞机飞行方向角调整的幅度不应超过 30°

$$-30\le x_i\le 30\cdots(i=1,2,3,4,5,6)$$

②惩罚项的添加

$$penality=10^6\left[\sum\max(0,-30-t_i)^2+\sum\max(0,t_i-30)^2\right]$$

若t_i超出边界，则添加大惩罚项。

③非线性不等式约束

$$b^2-4ac\le 0$$

其中，$a=4\sin^2\left(\frac{\theta_i-\theta_j}{2}\right)$，$b=2\left[(x_i-x_j)(\cos\theta_i-\cos\theta_j)+(y_i-y_j)(\sin\theta_i-\sin\theta_j)\right]$，$c=(x_i-x_j)^2+(y_i-y_j)^2-64$

（推导过程见第 2 题中“⑶②非线性不等式约束”部分）

⑷求解与输出

利用使用罚函数法迭代求解。每次迭代定义罚函数$penalizedObj(t)$，将约束违反的平方和乘以罚因子M加到目标函数上。

约束检查：计算$conFun(tNew)$，$violation=\mathrm{sum}\left(\max(0,ineq)\right)$表示约束违反的总和。若$violation>10^{-6}$，将当前解更新为$tNew$，罚因子$M$乘 10，增加对约束的强制性。若$violation\le 10^{-6}$，则停止迭代。

输出最优解（6×1 向量）、目标函数的最优值和$\mathrm{sum}\left(\max(0,ineq)\right)$——约束违反程度，若接近 0，则约束满足。

由于x_0是随机的，每次运行结果可能不同。

⑸代码实现和输出

```
clc; clear; close all;
% 定义优化目标函数
objFun = @(t) sum(t.^2);
% 初始值和边界
```

```
t0 = rand(6,1);
lb = -30 * ones(6,1);
ub = 30 * ones(6,1);
% 罚函数法参数
M = 100000;      % 初始罚因子
tol = 1e-6;     % 收敛容差
t = t0;         % 初始解
% 迭代求解
for k = 1:15
    % 定义带罚项的目标函数
    penalizedObj = @(t) objFun(t) + M * sum(max(0, conFun(t)).^2);
    % 使用无约束优化（fminunc）求解
    options = optimoptions('fminunc', 'Display', 'off');
    [tNew, fVal] = fminunc(@(t) boundPenalty(t, penalizedObj, lb, ub), t,
options);
    % 检查约束违反程度
    ineq = conFun(tNew);
    violation = sum(max(0, ineq));
    % 判断是否收敛
    if violation < tol
        break;
    end
    % 更新解和罚因子
    t = tNew;
    M = M * 10; % 增大罚因子
end
% 输出结果
disp('Optimal solution:');
disp(t);
disp('Objective function value:');
disp(objFun(t));
disp('Constraint violation:');
disp(sum(max(0, conFun(t))));
% ---- 局部函数：约束函数 ----
function g = conFun(t)
    th0 = [243 236 220.5 159 230 52]'; % 初始角度
    th = th0 + t;                     % 调整后角度
    x0 = [150 85 150 145 130 0]';     % x 坐标
    y0 = [140 85 155 50 150 0]';      % y 坐标
    min_distance = 8;                % 最小不碰撞距离
    g = zeros(15,1); % 预分配约束数组
    k = 1;
    for i = 1:5
        for j = i + 1:6
```

```
            a = 4 * (sind((th(i) - th(j)) / 2))^2;
            b = 2 * ((x0(i) - x0(j)) * (cosd(th(i)) - cosd(th(j))) + ...
                    (y0(i) - y0(j)) * (sind(th(i)) - sind(th(j))));
            c = (x0(i) - x0(j))^2 + (y0(i) - y0(j))^2 - min_distance^2;
            g(k) = b^2 - 4 * a * c;
            k = k + 1;
        end
    end
end
% ---- 局部函数：处理边界约束的罚函数 ----
function f = boundPenalty(t, penalizedObj, lb, ub)
    penalty = 1e6 * (sum(max(0, lb - t).^2) + sum(max(0, t - ub).^2));
    f = penalizedObj(t) + penalty;
end
```

由于x_0是随机的，每次运行结果可能不同，这里举出一个例子：

```
Optimal solution:
   -6.5393
    0.2744
    2.3039
   -0.4722
    6.4525
    1.3256

Objective function value:
   91.7602

Constraint violation:
   4.6657e-06
```

4.

3.4 求下列问题的解：

$$\max f(\boldsymbol{x}) = 2x_1 + 3x_1^2 + 3x_2 + x_2^2 + x_3,$$

$$\text{s.t.}\begin{cases} x_1 + 2x_1^2 + x_2 + 2x_2^2 + x_3 \leqslant 10, \\ x_1 + x_1^2 + x_2 + x_2^2 - x_3 \leqslant 50, \\ 2x_1 + x_1^2 + 2x_2 + x_3 \leqslant 40, \\ x_1^2 + x_3 = 2, \\ x_1 + 2x_2 \geqslant 1, \\ x_1 \geqslant 0, x_2, x_3 \text{ 无约束。} \end{cases}$$

⑴变量定义

三个变量x_1, x_2, x_3

两个初始点x_{0_1}, x_{0_2}，3×1 向量

⑵目标函数

最大化给定的函数：

$$\max f(x) = 2x_1 + 3x_1^2 + 3x_2 + x_2^2 + x_3$$

⑶约束条件

①边界限制$x_1 \geq 0$

②线性不等式约束$x_1 + 2x_2 \geq 1$

③非线性等式约束$x_1^2 + x_3 - 2 = 0$

④非线性不等式约束

$$\begin{cases} x_1 + 2x_1^2 + x_2 + 2x_2^2 + x_3 - 10 \leq 0 \\ x_1 + x_1^2 + x_2 + x_2^2 - x_3 - 50 \leq 0 \\ 2x_1 + x_1^2 + 2x_2 + x_3 - 40 \leq 0 \end{cases}$$

⑷求解与输出

利用使用 fmincon 函数法从两个初始点求解，并选择更优解输出。

⑸代码实现和输出

```
% 目标函数（取负以实现最大化）
fun = @(x) -(2*x(1) + 3*x(1)^2 + 3*x(2) + x(2)^2 + x(3));
% 线性不等式约束 A*x ≤ b
A = [-1, -2, 0];
b = -1;
% 变量下界
lb = [0; -inf; -inf];
% 初始点 1: x1=0, x2=0.5, x3=2
x0_1 = [0; 0.5; 2];
options = optimoptions('fmincon', 'Display', 'off', 'Algorithm', 'sqp');
[x1, fval1] = fmincon(fun, x0_1, A, b, [], [], lb, [], @nonlcon, options);
% 初始点 2: x1=1, x2=0, x3=1
x0_2 = [1; 0; 1];
[x2, fval2] = fmincon(fun, x0_2, A, b, [], [], lb, [], @nonlcon, options);
% 选择更优解
if -fval1 > -fval2
    x_opt = x1;
```

```
    f_opt = -fval1;
else
    x_opt = x2;
    f_opt = -fval2;
end
% 显示结果
disp('最优解:');
disp(['x1 = ', num2str(x_opt(1))]);
disp(['x2 = ', num2str(x_opt(2))]);
disp(['x3 = ', num2str(x_opt(3))]);
disp(['目标函数最大值: ', num2str(f_opt)]);
% 非线性约束函数
function [c, ceq] = nonlcon(x)
    c = [x(1) + 2*x(1)^2 + x(2) + 2*x(2)^2 + x(3) - 10;
         x(1) + x(1)^2 + x(2) + x(2)^2 - x(3) - 50;
         2*x(1) + x(1)^2 + 2*x(2) + x(3) - 40];
    ceq = x(1)^2 + x(3) - 2;
end
```

```
最优解:
x1 = 2.3333
x2 = 0.16667
x3 = -3.4444
目标函数最大值: 18.0833
```